

HR Analytics: Job Change of Data Scientists

Binary Classification | Model Comparison | Candidate Ranking System

Predicting which data-science trainees are likely to look for a new job, enabling targeted retention and hiring outreach.

Table of Contents

- Problem Statement
- Dataset
- Methodology
- Results
- Business Impact
- Project Structure
- Installation
- Usage
- Key Findings
- Next Steps

Problem Statement

A company offers free data-science training courses. Some trainees stay and build a career; others use the training as a stepping stone and are already looking to move elsewhere.

The challenge: Training every candidate is costly. The company needs to identify — in advance — which candidates are likely to job-hunt, so retention efforts and hiring outreach can be focused on the right people instead of applied uniformly.

This is framed as a **binary classification** problem:

- **target** = 1 → Looking for a new job
- **target** = 0 → Not looking

Beyond a single yes/no prediction, the project extends the model into a **candidate ranking system** that surfaces the Top 10 candidates most likely to be job-hunting — a prioritized shortlist for HR teams with limited capacity.

Dataset

Attribute	Value
Source	HR Analytics –Job Change of Data Scientists (Kaggle)
Training rows	19,158
Training columns	14 (13 features + target)
Test rows (unlabeled)	2,129

Table 1 – continued

Attribute	Value
Target	1 = looking for a new job, 0 = not looking
Class balance	75.1% class 0 / 24.9% class 1 (imbalanced)

Data Quality Issues Handled

- **Missing values** (up to 32% in `company_type`, `company_size`, `gender`, `major_discipline`) — kept as their own category since missingness carries predictive signal
- **Disguised numeric text** (experience: ‘<1’, ‘>20’; last_new_job: ‘never’, ‘>4’) — mapped to real integers
- **High-cardinality city** (123 unique values) — dropped in favor of `city_development_index`
- **enrollee_id** — unique identifier with no predictive value, dropped
- **Ordinal columns** (`education_level`, `company_size`) — encoded preserving order
- **Nominal columns** (`gender`, `relevant_experience`, `enrolled_university`, `major_discipline`, `company_type`) — one-hot encoded
- **Engineered feature**: flag for candidates missing both `company_type` and `company_size` → correlates strongly with job-hunting (2× higher target rate)

Methodology

1. Preprocessing Pipeline

- Cleaned and encoded per the issues above
- Fully numeric feature matrix with zero missing values
- Stratified 80/20 train/validation split (preserves 75/25 class balance)

2. Models Trained

Five classification algorithms were compared:

Model	Feature Scaling	Class Imbalance Handling
Logistic Regression	✓ Standardized	<code>class_weight='balanced'</code>
K-Nearest Neighbors (KNN)	✓ Standardized	—
Support Vector Machine (SVM)	✓ Standardized	<code>class_weight='balanced'</code>
Random Forest	✗ Raw features	<code>class_weight='balanced'</code>
XGBoost	✗ Raw features	<code>scale_pos_weight</code>

3. Threshold Tuning

- Evaluated **19 thresholds** (0.05 to 0.95) per model
- Selected the threshold maximizing **F1-score** for each model
- Default 0.5 threshold is suboptimal on imbalanced data

4. Evaluation Metric

F1-score (harmonic mean of precision and recall) — balances catching true job-seekers (recall) against not over-flagging stable candidates (precision).

🏆 Results

Model Comparison (Best Threshold per Model)

Figure 1: F1 Score Across 19 Thresholds

Rank	Model	Best Threshold	F1-Score	Accuracy
🥇 1	XGBoost	0.50	0.6364	0.7868
🥈 2	SVM	0.35	0.6201	0.7672
🥉 3	Random Forest	0.30	0.6138	0.7714
4	Logistic Regression	0.55	0.6006	0.7539
5	KNN	0.05	0.5029	0.6203

Final Model: XGBoost

Figure 2: Classification Report & Confusion Matrix

Class	Precision	Recall	F1-Score	Support
0 (Not looking)	0.91	0.80	0.85	2,877
1 (Looking)	0.55	0.75	0.64	955
Accuracy			0.79	3,832

Confusion Matrix:

- True Negatives: 2,300 | False Positives: 577
- False Negatives: 240 | True Positives: 715

The model correctly identifies **715 of 955** actual job-seekers (Recall = 0.75) at the cost of 577 false positives. Prioritizing recall is preferable here — missing a real job-seeker is more costly than a follow-up with a stable candidate.

Cross-Validation (F1 Reliability Check)

Fold	F1-Score
1	0.6118
2	0.6056
3	0.6207
4	0.6164
5	0.6087

Mean F1 across folds: 0.613 ± 0.005

Low standard deviation indicates stable, reliable performance. The cross-validated F1 (0.61) is the more defensible number — slightly below the single-split 0.636, as expected.

Candidate Ranking System (Precision@k)

k	Precision@k	Correct / k
10	0.60	6 / 10
20	0.60	12 / 20
50	0.66	33 / 50
100	0.61	61 / 100

Precision@k values of 0.60–0.66 are 2.4–2.6× higher than random guessing baseline (~0.25), confirming the ranking is genuinely informative.

 Business Impact

Metric	Value	Meaning
2.6×	Precision@50 = 0.66 vs. 0.25 baseline	Far better than random selection
75%	Recall on job-seekers	Catches 3 out of 4 real job-hunters
Top-10 List	Precision@10 = 0.60	Actionable shortlist for limited HR capacity

Key Insight

Candidates missing both company_type and company_size are 2× more likely to be job-hunting — a strong signal that these candidates may not be currently employed.

 Project Structure

```
hr-analytics-job-change/  
├─ data/
```

```
|   ├── train.csv          # Original training data (19,158 rows)
|   └── test.csv           # Unlabeled test data (2,129 rows)
├── notebooks/
|   ├── 01_eda.ipynb       # Exploratory Data Analysis
|   ├── 02_preprocessing.ipynb # Data cleaning & feature engineering
|   ├── 03_modeling.ipynb  # Model training & comparison
|   └── 04_evaluation.ipynb # Threshold tuning, CV, ranking system
├── src/
|   ├── preprocess.py      # Preprocessing pipeline
|   ├── models.py          # Model definitions & training
|   ├── evaluate.py        # Evaluation & threshold tuning
|   └── rank_candidates.py  # Candidate ranking system
├── images/
|   ├── threshold_comparison.png # F1 across 19 thresholds (5 models)
|   └── final_results.png      # XGBoost confusion matrix & report
├── reports/
|   ├── HR_Analytics_Project_Report.docx
|   └── HR_Analytics_Presentation.pptx
├── requirements.txt
└── README.md
```

Installation

```
# Clone the repository
git clone https://github.com/yourusername/hr-analytics-job-change.git
cd hr-analytics-job-change

# Create virtual environment
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt
```

Requirements

```
pandas>=1.5.0
numpy>=1.24.0
scikit-learn>=1.3.0
xgboost>=2.0.0
matplotlib>=3.7.0
seaborn>=0.12.0
jupyter>=1.0.0
```

Usage

1. Run the full pipeline

```
python src/main.py
```

2. Or step-by-step in Jupyter

```
jupyter notebook notebooks/
```

3. Generate candidate rankings

```
from src.rank_candidates import rank_candidates

# Load trained model and data
model = load_model('models/xgboost_final.pkl')
X_test = preprocess(test_df)

# Get top 10 candidates most likely to job-hunt
top_10 = rank_candidates(model, X_test, k=10)
print(top_10)
```

Key Findings

1. **XGBoost wins** — Highest F1 (0.636 single split; 0.613 ± 0.005 CV) and best ROC-AUC among all five models
2. **Class imbalance handling is critical** — Default unweighted models scored F1 = 0.00 on the minority class
3. **Threshold tuning matters** — Best thresholds varied: XGBoost at 0.50, SVM at 0.35, Random Forest at 0.30
4. **Missing company info = strong signal** — Engineered feature doubled predictive power for that segment
5. **Deep learning underperforms** — PyTorch neural network (F1 = 0.591) lost to gradient-boosted trees on this tabular dataset (~15K rows)
6. **Ranking system delivers real value** — 2.4–2.6× better than random, giving HR an actionable prioritized list

Next Steps

- **Hyperparameter tuning** — Grid/Random search beyond default XGBoost config
 - **Interaction features** — e.g. `experience × company_size`
 - **Model refresh pipeline** — Periodic retraining as new candidate cohorts arrive
 - **A/B testing** — Validate ranking system against current HR workflow
 - **Deploy API** — Serve real-time predictions for the HR dashboard
-

License

This project is licensed under the MIT License.

Acknowledgments

- Dataset: [Kaggle — HR Analytics Job Change of Data Scientists](#)
- Built with scikit-learn, XGBoost, and Python

Author: [Your Name](#)

Date: August 2026